

Maps and sets

A map pairs keys with values. The literal uses the same square brackets as a list, with a colon between key and value.

```
use std.io

fn main() {
    var stock = ["apples": 12, "pears": 3]
    stock.set("plums", 7)

    io.say("{stock.get("apples") ?? 0} apples")
    io.say("has pears: {stock.has("pears")}")
    io.say("{stock.len()} kinds of fruit")
    io.say("keys {stock.keys()}")
}
```

`get` gives back "the value, or nothing", which is why `?? 0` is there: it says what to use when the key is missing. The compiler will not let you forget.

An empty map is written `[:]` so it cannot be confused with an empty list.

Walking a map

```
use std.io

fn main() {
    let ages = ["ada": 36, "alan": 41, "grace": 45]
    for name in ages.keys() {
        io.say("{name.pad_right(8)} {ages.get(name) ?? 0}")
    }
}
```

Counting things

A map is the natural way to count.

```
use std.io

fn main() {
    var counts: Map<Text, Int> = [:]
    for word in "the cat sat on the mat the end".split(" ") {
        counts.set(word, (counts.get(word) ?? 0) + 1)
    }
    for word in counts.keys() {
        io.say("{word}: {counts.get(word) ?? 0}")
    }
}
```

Sets

A set holds each value once and answers "is this in here" quickly.

```
use std::io

fn main() {
    var seen = set([1, 2, 3, 2, 1])
    seen.add(9)
    io.say("{seen.len()} unique values")
    io.say("has 9: {seen.has(9)}")
    io.say("as a list: {seen.items()}")
}
```

Exercise

Read a sentence and report the three most common words. `sort_by` on the key list will help.